TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Universidad Tecnológica Nacional

Programación 1

Sistema de Control de Inventario

Informe de diseño y fundamentación

Trabajo Práctico Integrador: ferretería, gestión de herramientas y stock en Python

Alumno/a: Martin Ricciardi

Materia: Programación 1

Comisión: 13

Entrega: Noviembre de 2026

Índice

1. Introducción……3
1.1 Objetivos……3
2. Lenguaje, entorno y recursos de desarrollo……3
2.1 Lenguaje: Python……3
2.2 Entorno: Visual Studio Code……3
2.3 Intérprete y ejecución……3
2.4 Organización de los archivos……3
3. Diseño de la solución……4
3.1 Requerimientos……4
3.2 Estructura de datos……4
3.3 Arquitectura modular……4
3.4 Criterios de validación……5
3.5 Transferencia de datos entre módulos sin variables globales……5
4. Conceptos aplicados…… 5
4.1 Listas……5
4.2 Diccionarios……6
4.3 Funciones……6
4.4 Condicionales……6
4.5 Bucles……6
4.6 Manejo de errores con try/except y raise……7
5. Validaciones implementadas……7
6. Diagramas de flujo……9
7. Cumplimiento de las reglas técnicas…… 13
8. Declaración de transparencia uso de IA……13
9. Conclusiones……13
Referencias……13

1. Introducción

Una ferretería local necesita digitalizar el control de sus productos para evitar pérdidas de stock. Hoy maneja sus datos de forma manual y requiere un programa que permita gestionar las herramientas a la venta y sus unidades disponibles. Este informe documenta el diseño y la fundamentación de la solución desarrollada: un programa en Python con menú interactivo por consola.

1.1 Objetivos

- Representar información real con una lista de diccionarios.
- Dividir el programa en funciones, cada una con una tarea clara.
- Usar condicionales y bucles para controlar el flujo de un programa interactivo.
- Lograr que el programa resista los errores del usuario mediante `try/except` y `raise`.
- Documentar el diseño de la solución y vincular cada concepto con una parte concreta del código.

2. Lenguaje, entorno y recursos de desarrollo

2.1 Lenguaje: Python

Python es un lenguaje de programación interpretado, de alto nivel y de propósito general, cuya sintaxis prioriza la legibilidad. Se utilizó **Python 3.13.1**, versión con la que se ejecutó y verificó el programa. No se emplean bibliotecas externas: solo funciones incorporadas (`input`, `print`, `int`, `len`, `range`) y métodos de texto (`strip()`, `lower()`). Se eligió Python porque es el lenguaje de la asignatura, porque sus listas y diccionarios incorporados permiten representar el inventario sin dependencias, y porque su manejo de excepciones (`try/except` y `raise`) cubre los requisitos de robustez del trabajo.

2.2 Entorno: Visual Studio Code

Visual Studio Code es un editor de código fuente gratuito y multiplataforma desarrollado por Microsoft, con resaltado de sintaxis, terminal integrada y extensiones; con la extensión de Python permite ejecutar y depurar programas. Versión utilizada: **1.145.1**. Se eligió porque el enunciado solicita mostrar el código en este editor y porque permite editar los tres archivos del proyecto y ejecutarlos desde su terminal integrada.

2.3 Intérprete y ejecución

El programa se ejecuta desde la terminal, ubicada en la carpeta que contiene los tres archivos, con el comando `python main.py`. El intérprete de Python traduce y ejecuta el código línea a línea, por lo que no hace falta compilar. Se eligió esta forma de ejecución porque es la más simple y no depende de ninguna herramienta adicional.

2.4 Organización de los archivos

El programa se divide en tres archivos **.py**. Para iniciar el sistema se ejecuta **`main.py`**; los otros dos son módulos que `main.py` importa y deben estar en la misma carpeta.

Archivo	Contenido	Rol
main.py	Función `main()` con el bucle del menú y la opción 7.	Programa principal (se ejecuta)
opciones_menu.py	Las funciones de las opciones 1 a 6 del menú.	Módulo importado
auxiliares.py	Funciones de lectura, validación, búsqueda y menú.	Módulo importado

3. Diseño de la solución

3.1 Requerimientos

El programa presenta un menú persistente con siete opciones: después de cada operación el menú vuelve a mostrarse hasta que el usuario elige la opción 7. Cada opción se resuelve en una función propia, salvo la opción 7, que termina el bucle del menú en `main()`.

Opción	Funcionalidad	Función	Módulo
1	Carga de herramientas con existencias iniciales (carga múltiple)	`cargar_herramientas`	opciones_menu.py
2	Visualización de inventario	`mostrar_inventario`	opciones_menu.py
3	Consulta de stock por nombre	`consultar_stock`	opciones_menu.py
4	Reporte de agotados (stock igual a cero)	`reporte_agotados`	opciones_menu.py
5	Alta de nuevo producto	`alta_producto`	opciones_menu.py
6	Actualización de stock (venta / ingreso)	`actualizar_stock`	opciones_menu.py
7	Salir	se resuelve al terminar el bucle	main.py

3.2 Estructura de datos

El inventario se representa con una única lista de diccionarios llamada `inventario`. Cada elemento es una herramienta, y todos los diccionarios tienen exactamente las mismas dos claves: `'herramienta'` (el nombre) y `'cantidad'` (el stock, es decir, las unidades disponibles). Con dos herramientas, 10 martillos y 3 palas, la estructura es:

```
[{'herramienta': 'Martillo', 'cantidad': 10}, {'herramienta': 'Pala', 'cantidad': 3}]
```

Se eligió esta estructura porque reúne en un mismo elemento los dos datos de cada herramienta, conserva el orden de carga (las altas se agregan al final) y permite obtener cualquier resultado recorriendo la lista, sin necesidad de otras colecciones.

3.3 Arquitectura modular

El código se organiza en tres módulos con responsabilidades separadas. `main.py` depende de `opciones_menu.py` y de `auxiliares.py`; `opciones_menu.py` depende de `auxiliares.py`; y `auxiliares.py` no depende de ningún otro. Así, si hay un error, se sabe en qué función buscarlo.

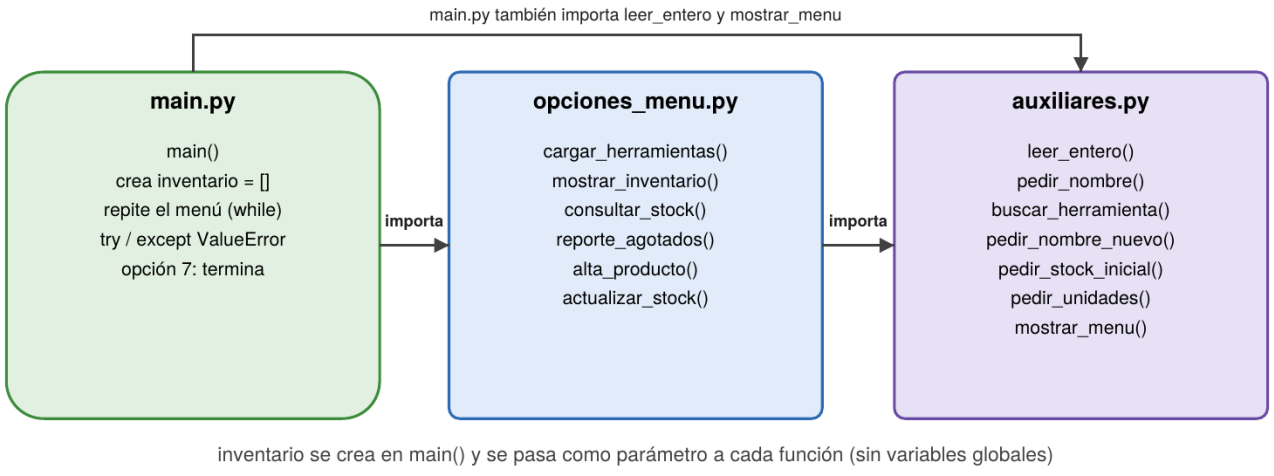


Figura 1. Arquitectura modular del programa y dependencias entre archivos.

3.4 Criterios de validación

La siguiente tabla resume el rol de cada función auxiliar y de las funciones de menú, e indica cuáles lanzan `ValueError`.

Función	Responsabilidad	Lanza ValueError
`leer_entero(mensaje)`	Pide un dato y lo convierte a entero. Si `int()` falla, relanza el error con un mensaje claro.	Si no es entero
`pedir_nombre()`	Pide un nombre y lo devuelve sin espacios al inicio ni al final.	Si está vacío
`buscar_herramienta (inventario, nombre)`	Busca ignorando mayúsculas y espacios extremos. Devuelve el diccionario existente o `None`.	No
`pedir_nombre_nuevo(inventario)`	Pide un nombre válido que no esté repetido.	Vacío o duplicado
`pedir_stock_inicial()`	Pide el stock inicial (entero, mayor o igual que 0).	No entero o negativo
`pedir_unidades()`	Pide las unidades de un movimiento (entero mayor que 0).	No entero o menor o igual que 0
`mostrar_menu()`	Imprime las siete opciones.	No
`cargar_herramientas(inventario)`	Opción 1. Carga múltiple; captura sus errores y reintenta.	No (los captura en el bucle)
`mostrar_inventario(inventario)`	Opción 2. Lista las herramientas con su stock.	No
`consultar_stock(inventario)`	Opción 3. Muestra el stock de una herramienta.	Nombre vacío
`reporte_agotados(inventario)`	Opción 4. Lista las de stock igual a cero.	No
`alta_producto(inventario)`	Opción 5. Agrega una herramienta al final.	Cualquier dato inválido
`actualizar_stock(inventario)`	Opción 6. Venta o ingreso de unidades.	Datos inválidos o stock insuficiente
`main()`	Crea el inventario, repite el menú y captura los errores que llegan.	Lo lanza para opción fuera de rango

3.5 Transferencia de datos entre módulos sin variables globales

`inventario` se crea una sola vez, dentro de `main()`, y se pasa como parámetro a cada función que lo necesita. No hay variables globales. Las listas y los diccionarios son tipos mutables: al pasar la lista por parámetro, la función recibe una referencia a la misma lista y no una copia. Por eso un `append()` o un cambio en `producto['cantidad']` hecho dentro de una función se ve después en `main()` sin necesidad de `return`.

4. Conceptos aplicados

Cada concepto se define brevemente y se vincula con la parte concreta del programa donde se aplica.

4.1 Listas

Una lista es una colección ordenada y modificable de elementos. En el programa, `inventario` es la única lista: se crea vacía en `main()`; crece con `append()` en `cargar_herramientas` (opción 1) y en `alta_producto` (opción 5), siempre al final; se consulta su tamaño con `len(inventario)` para detectar que todavía no hay herramientas cargadas (opciones 2, 3, 4 y 6); y se recorre con `for` en `buscar_herramienta`, `mostrar_inventario` y `reporte_agotados`. No se usan otras listas: por ejemplo, el reporte de agotados se obtiene recorriendo `inventario` y no armando una lista auxiliar.

```
inventario = []                                  # main(): se crea una sola vez
inventario.append({'herramienta': nombre, 'cantidad': stock})   # opciones 1 y 5
if len(inventario) == 0:                         # opciones 2, 3, 4 y 6
    print('Todavía no hay herramientas cargadas.')
```

4.2 Diccionarios

Un diccionario asocia claves con valores. Cada herramienta es un diccionario con las claves `'herramienta'` y `'cantidad'`, que se leen con `producto['herramienta']` y `producto['cantidad']`. El diccionario de una herramienta nueva se arma directamente dentro de `inventario.append(...)`, sin crear diccionarios sueltos. En `buscar_herramienta`, la variable `producto` apunta a un diccionario que ya pertenece a la lista; por eso `actualizar_stock` puede ejecutar `producto['cantidad'] -= unidades` y modificar el inventario real.

```
def buscar_herramienta(inventario, nombre):
    nombre_buscado = nombre.strip().lower()
    for producto in inventario:
        if producto['herramienta'].lower() == nombre_buscado:
            return producto          # el diccionario que ya está dentro de inventario
    return None
```

4.3 Funciones

Una función agrupa instrucciones bajo un nombre, recibe parámetros y puede devolver un resultado con `return`. El programa define 14 funciones con una tarea clara cada una: 7 auxiliares, 6 de menú y `main()`. Las funciones reciben `inventario` por parámetro y algunas devuelven valores: `leer_entero` devuelve un entero, `pedir_nombre` devuelve un texto y `buscar_herramienta` devuelve el diccionario encontrado o `None`. La modularización evita repetir código: `pedir_nombre_nuevo` y `pedir_stock_inicial` se usan tanto en la opción 1 como en la 5, de modo que la regla de duplicados y la de stock no negativo se escriben una sola vez. Los nombres siguen la convención `snake_case`.

```
def pedir_nombre_nuevo(inventario):
    nombre = pedir_nombre()
    if buscar_herramienta(inventario, nombre) is not None:
        raise ValueError(f'La herramienta "{nombre}" ya existe en el inventario.')
    return nombre
```

4.4 Condicionales

Los condicionales `if`, `elif` y `else` ejecutan código según se cumpla o no una condición. Se aplican en tres tipos de decisiones. Primero, la selección de la opción del menú en `main()` mediante una cadena `if/elif`. Segundo, las reglas de negocio: un `if` comprueba la regla (nombre vacío, stock negativo, unidades menores o iguales que 0, venta mayor que el stock) y, si se incumple, `raise` lanza el error. Tercero, los casos informativos (inventario vacío, herramienta inexistente, tipo de movimiento inválido), que se resuelven con un `if`, un mensaje y un `return` sin excepción. En la opción 6, el tipo de movimiento se compara como texto, sin convertirlo con `int()`:

```
tipo = input('Elija el tipo de movimiento: ').strip().lower()
es_venta = tipo == '1' or tipo == 'venta'
es_ingreso = tipo == '2' or tipo == 'ingreso'
if not es_venta and not es_ingreso:
    print('Tipo de movimiento inválido: solo se acepta venta o ingreso.')
    return
```

4.5 Bucles

Los bucles repiten instrucciones. El programa usa tres formas. El bucle `while opcion != 7` de `main()` mantiene el menú activo hasta que se elige salir. Los bucles `while True` con `break` de `cargar_herramientas` repiten la petición de un dato mientras sea inválido: el `except` solo captura el error, y es el bucle el que organiza el reintento. El bucle `for numero in range(1, total + 1)` repite la carga tantas veces como herramientas indicó el usuario. Por último, `for producto in inventario` recorre la lista en `buscar_herramienta`, `mostrar_inventario` y `reporte_agotados`.

```
nombre = ''
while True:                                  # reintenta hasta que el nombre sea válido
    try:
        nombre = pedir_nombre_nuevo(inventario)
        break                                # nombre válido: sale del bucle
    except ValueError as e:
        print(f'Error: {e}')                # informa y vuelve a pedir
```

4.6 Manejo de errores con try/except y raise

El programa distingue dos situaciones. Si el usuario escribe “abc” donde se espera un número, es una **entrada inválida**: `int()` falla con `ValueError` y se captura con `try/except`. Si escribe un nombre que ya existe, el texto es válido pero viola una **regla de negocio**, y la función de validación la lanza explícitamente con `raise ValueError` y un mensaje descriptivo. `ValueError` es una excepción incorporada en Python, por lo que no hace falta definir clases propias.

En `leer_entero`, el error de `int()` se captura y se vuelve a lanzar con un mensaje claro, de modo que todas las conversiones del programa muestran el mismo texto comprensible:

```
def leer_entero(mensaje):
    try:
        return int(input(mensaje))
    except ValueError:
        raise ValueError('Debe ingresar un número entero.')
```

Dónde se captura cada error depende de la operación. En la opción 1, la función captura el error y reintenta el dato. En las opciones 3, 5 y 6, la función no lo captura: el error sale de la función, llega al `except` de `main()`, que muestra el motivo y vuelve al menú.

```
try:
    opcion = leer_entero('Seleccione una opción: ')
    if opcion == 1:
        cargar_herramientas(inventario)
    elif opcion == 2:
        mostrar_inventario(inventario)
    # ... opciones 3 a 6 con el mismo esquema
    elif opcion != 7:
        raise ValueError('Opción fuera de rango (1 a 7).')
except ValueError as e:
    print(f'Error: {e}')
```

Un dato inválido nunca deja datos a medias. En la opción 1, la herramienta se agrega con `append()` recién cuando el nombre y el stock son válidos; en la opción 5, el stock se pide solo si el nombre ya fue aceptado; y en la opción 6, el stock se modifica en el último paso, después de superar todas las validaciones. Por eso un intento inválido no modifica ningún stock.

5. Validaciones implementadas

La siguiente tabla resume cada situación de validación, el mecanismo utilizado, la función donde se resuelve y qué ocurre después. Los nombres se comparan sin distinguir mayúsculas ni espacios al inicio y al final, de modo que “Martillo” y “ martillo ” son la misma herramienta; el nombre se guarda sin esos espacios y respetando las mayúsculas escritas.

Situación	Mecanismo	Función	Qué ocurre después
Nombre vacío o solo espacios	`raise ValueError`	`pedir_nombre`	Opción 1: se pide de nuevo. Opciones 3, 5 y 6: vuelve al menú
Nombre duplicado	`raise ValueError`	`pedir_nombre_nuevo`	Opción 1: se pide de nuevo. Opción 5: vuelve al menú, sin pedir el stock
Dato no convertible a entero	`try/except`	`leer_entero`	Opción 1: reintenta. Opciones 5 y 6 y menú: vuelve al menú
Stock inicial negativo (el 0 es válido)	`raise ValueError`	`pedir_stock_inicial`	Opción 1: se pide solo el stock, conservando el nombre. Opción 5: vuelve al menú
Cantidad de herramientas a cargar menor o igual que 0	`raise ValueError`	`cargar_herramientas`	Se informa y se pide de nuevo ese dato
Unidades de venta o ingreso menores o iguales que 0	`raise ValueError`	`pedir_unidades`	Vuelve al menú; el stock no cambia
Venta que supera el stock disponible	`raise ValueError`	`actualizar_stock`	Vuelve al menú; el stock no cambia
Opción del menú fuera de 1 a 7	`raise ValueError`	`main`	Vuelve al menú
Herramienta inexistente o inventario vacío	Condición y mensaje	opciones 2, 3, 4 y 6	Vuelve al menú sin modificar nada
Tipo de movimiento distinto de venta o ingreso	Condición y mensaje	`actualizar_stock`	Vuelve al menú sin modificar nada

6. Diagramas de flujo

Los diagramas de esta sección son propios y reflejan el código entregado. Los rombos son decisiones, los rectángulos redondeados azules son procesos, los violetas son operaciones sobre `inventario`, los rojos son errores, los grises son casos informativos y los verdes marcan inicio o fin. El primero muestra el flujo del menú principal.

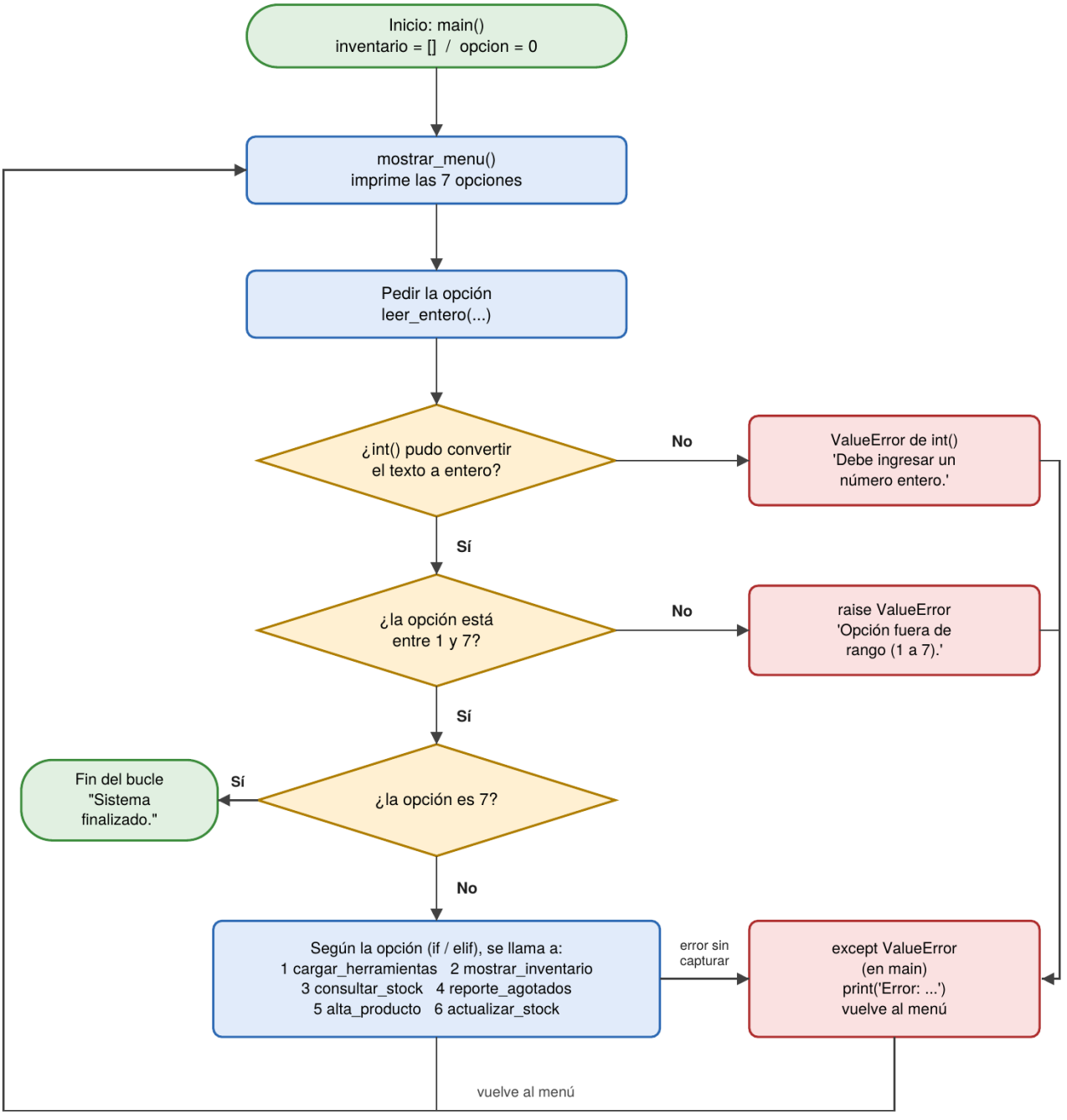


Figura 2. Flujo del menú principal en main(): lectura, validación y despacho de la opción.

El siguiente diagrama muestra el recorrido de la carga múltiple de la opción 1, con sus tres reintentos dentro de la propia función: cantidad, nombre y stock.

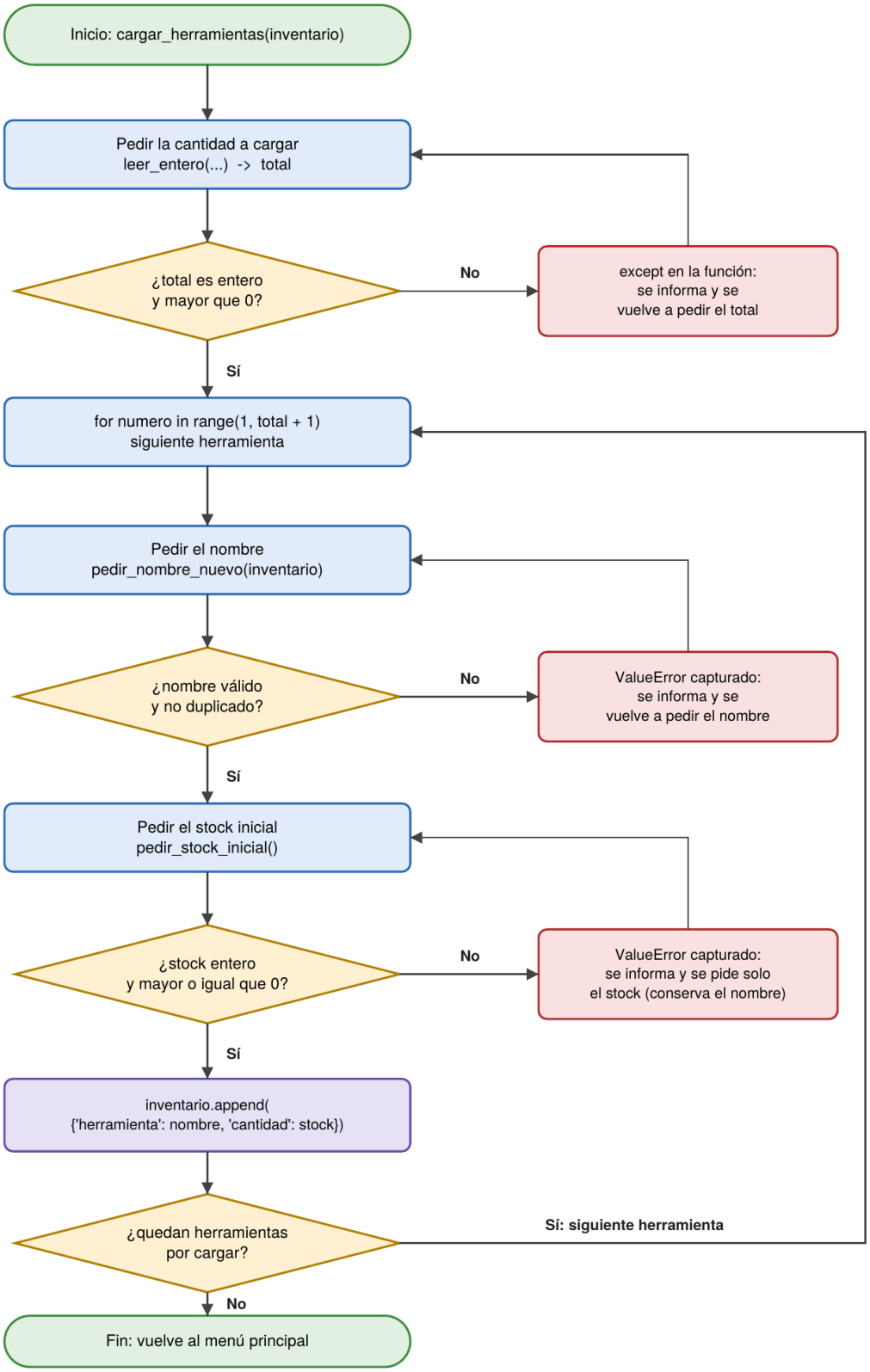


Figura 3. Recorrido de la carga de herramientas (opción 1) con reintento de cada dato.

Este diagrama resume el recorrido de la validación de datos: primero la conversión a entero, luego la regla de negocio, y por último dónde se captura el error según la opción.

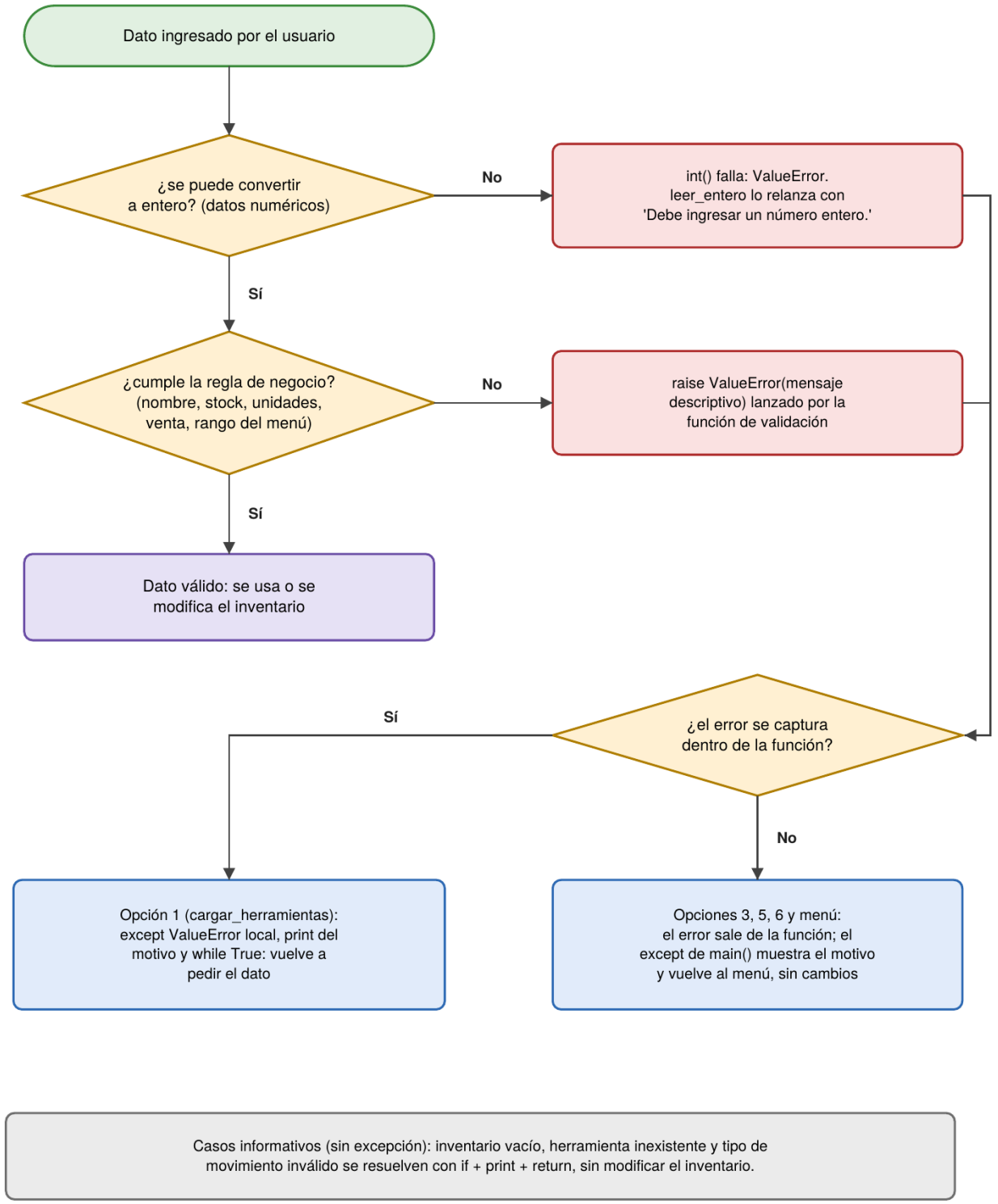


Figura 4. Recorrido de la validación de datos y destino de cada tipo de error.

Por último, el recorrido de la actualización de stock (opción 6), que modifica el stock solo al final.

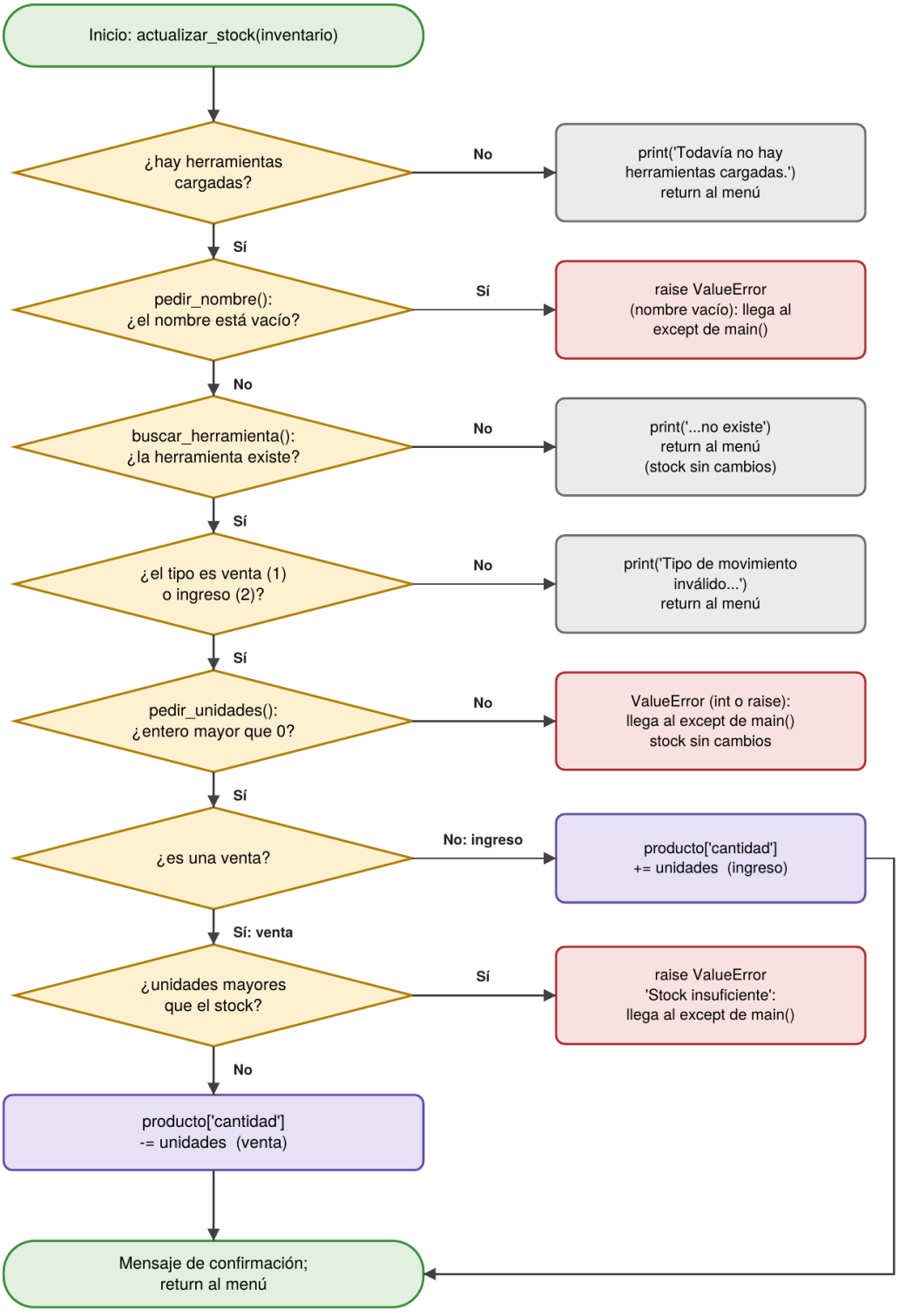


Figura 5. Recorrido de la actualización de stock (opción 6): venta e ingreso.

7. Cumplimiento de las reglas técnicas

Regla	Cómo se cumple en el programa
2.1 Estructuras permitidas	Solo se usan estructuras secuenciales, `if/elif/else`, `while`, `for` con `break`, funciones con `def` y `return`, y `try/except` con `raise`. Se divide el programa en tres archivos .py propios.
2.2 Estructura de datos	Una lista de diccionarios llamada `inventario`, con exactamente las claves `'herramienta'` y `'cantidad'`.
2.3 Modularización	Las opciones 1 a 6 tienen su función propia; la opción 7 se resuelve en `main()`.
2.4 Manejo de errores	Se usa `try/except` para las conversiones y `raise ValueError` para las reglas de negocio, con mensajes descriptivos.
2.5 Prohibiciones	No hay clases, ni diccionarios sueltos, ni otras colecciones fuera de `inventario`.
2.6 Variables globales	No hay variables globales: `inventario` se crea en `main()` y se pasa por parámetro.

8. Declaración de transparencia uso de IA

Declaro que, para la elaboración de este trabajo, utilicé la herramienta de inteligencia artificial Claude (Anthropic) como apoyo. Su uso comprendió los siguientes aspectos:

- **Estructuración del informe:** organización de las secciones y del orden en que se presentan los contenidos.
- **Desarrollo de ideas:** exploración de los contenidos a incorporar para abarcar lo que solicitaba la consigna, como los conceptos aplicados, las validaciones, el diagrama de flujo, los recursos de desarrollo y algunas propuestas como incluir bloques de código en las explicaciones o incorporar tablas para mostrar un trabajo más prolijo.
- **Corrección de ortografía y vocabulario académico:** revisión de la redacción y mejora de la terminología empleada.
- **Generación de imágenes:** elaboración de los diagramas de flujo incluidos en el informe junto a su pie de imagen.
- **Diseño del código y comprensión de la consigna:** asistencia en la creación del diseño del programa y en el correcto entendimiento de la consigna sobre el proyecto. Colaborando para poder estructurar una base con todos los requisitos planteados.

Todo el contenido, fue revisado por mí como autor del trabajo. Me hago responsable del producto final del informe y de su contenido.

9. Conclusiones

La separación en módulos permitió que cada función tenga una única tarea, que las reglas de validación se escriban una sola vez y que sea sencillo ubicar un error. La estructura de lista de diccionarios resultó suficiente para representar el inventario y para obtener todos los resultados recorriéndolo.

El diseño del manejo de errores fue el aspecto más formativo: distinguir una entrada inválida de una regla de negocio, decidir dónde capturar cada error (en la función para reintentar, o en `main()` para volver al menú) y comprender que pasar la lista por parámetro comparte la misma referencia.

Referencias

- Tecnicatura Universitaria en Programación a Distancia. *TPI Programación 1: Sistema de Control de Inventario (enunciado del trabajo)*. Guia para desarrollo del trabajo general.
- Tecnicatura Universitario en Programación a Distancia. *Material de la materia Programación 1*. Guia para desarrollo de código.
- Python Software Foundation. *PEP 8: Style Guide for Python Code*. https://peps.python.org/pep-0008/. Guia para uso de operadores de comparación en el desarrollo del código.